ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 4

TECHNICAL PROGRAMMING

PRACTICE EXAM 5 (PAPER 1) · MARKING GUIDELINE

MARKS: 120

Marking guideline for the Academic Central NC(V) Level 4 Technical Programming practice exam 5 (paper 1).

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 4. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Code is not marked strictly on syntax: give marks for correct logic and structure. Accept other correct answers, variable names and wording.

SECTION A

QUESTION 1

1 1.1 C ✓ 1.2 A ✓ 1.3 D ✓ 1.4 B ✓ 1.5 A ✓ 1.6 C ✓ 1.7 B ✓ 1.8 D ✓ 1.9 A ✓ 1.10 B ✓ (2 + 4 + 6 = 12) (10)

[10]

QUESTION 2

2 2.1 False ✓ 2.2 True ✓ 2.3 True ✓ 2.4 False ✓ 2.5 True ✓ 2.6 False ✓ 2.7 True ✓ 2.8 False ✓ 2.9 True ✓ 2.10 True ✓ (10)

[10]

QUESTION 3

3 3.1 non-volatile ✓ 3.2 multitasking ✓ 3.3 testing ✓ 3.4 flowchart ✓ 3.5 selection ✓ 3.6 square brackets ✓ 3.7 encrypted ✓ 3.8 Raspberry Pi OS ✓ 3.9 push ✓ 3.10 standalone ✓ (10)

[10]

QUESTION 4

4 4.1 E ✓ 4.2 C ✓ 4.3 G ✓ 4.4 A ✓ 4.5 F ✓ 4.6 D ✓ 4.7 J ✓ 4.8 H ✓ 4.9 B ✓ 4.10 I ✓ (10)

[10]

TOTAL SECTION A: [40]

SECTION B

QUESTION 5

5.1 More of the running programs and their data fit in fast RAM at the same time ✓, so the computer does not have to keep swapping data to the much slower storage drive ✓. (2)

5.2 Definition: standalone – installed and runs on the computer itself ✓; web app – runs on a server and is used through a browser ✓. Platform: standalone – made for a specific operating system ✓; web app – works on any device with a browser ✓. Internet: standalone – usually not needed ✓; web app – needs a connection ✓. (6)

5.3.1 In order ✓✓✓ (½ each): analysis; design; coding; testing; implementation; maintenance. (3)

5.3.2 Analysis works out WHAT the program must do (the problem, inputs, outputs and requirements) ✓; design plans HOW it will be done (algorithms, flowcharts, pseudocode, screens) ✓. (2)

5.3.3 Finding ✓ and fixing errors (bugs) in a program ✓. (2)

5.4.1 Whether the letter is a (lower-case) vowel ✓ (1)

5.4.2 ROBOT has 2 vowels ✓✓ (2)

5.5 Any TWO ✓✓: low power use (longer battery life); produce little heat; small and cheap; good performance per watt; widely supported. (2)

5.6 git clone <repository URL> ✓ (1)

5.7 It shows the history of commits ✓ – who made each change, when, and the commit message ✓. (2)

[23]

QUESTION 6

6.1 It loops through the list and gives a counting number with each item ✓, starting at 1 instead of 0 ✓. (2)

6.2 strip() removes spaces at the start and end, and lower() changes the text to lower case ✓, so "Motor " and "motor" are treated as the same part ✓. (2)

6.3 "Motor" becomes "motor" ✓, which is already in the list, so it displays "motor is already in the list." and nothing is added ✓. (2)

6.4 It tests another condition when the previous if / elif conditions are false ✓. (1)

6.5 Parts in stock: ✓ 1 motor ✓ 2 sensor ✓ 3 wheel 4 gripper ✓ (4)

6.6 def count_parts(): ✓ with a print ✓ that uses len(parts) ✓ and a clear message ✓, e.g. (4)

```
def count_parts():
    print("There are", len(parts), "parts in stock.")
```

6.7 When the user enters 4 ✓, the break statement ends the while True loop ✓. (2)

[17]

QUESTION 7

7.1 Correct dictionary name and curly braces ✓; four correct key–value pairs ✓✓✓ (4)

```
battery_levels = {'R1': 85, 'R2': 40, 'R3': 15, 'R4': 60}
```

7.2 print("R3 battery level:", battery_levels['R3'], "%") ✓✓ (2)

7.3 Input of the name ✓; input of the level as int ✓; battery_levels[name] = level ✓✓ (4)

```
name = input("New robot name: ")
level = int(input("Battery level: "))
battery_levels[name] = level
```

7.4 Input of the name ✓✓; if name in battery_levels ✓✓; del battery_levels[name] (or pop) ✓✓; message ✓; else "Robot not found" ✓ (8)

```
name = input("Robot taken out of service: ")
if name in battery_levels:
    del battery_levels[name]
    print(name, "removed.")
else:
    print("Robot not found")
```

7.5 for loop over the items ✓; if level < 20 ✓ print CHARGE ✓, else OK ✓ (4)

```
for robot, level in battery_levels.items():
    if level < 20:
        print(robot, "CHARGE")
    else:
        print(robot, "OK")
```

[22]

QUESTION 8

8.1 Keeping an object's data (attributes) and the methods that work on it together inside a class ✓, and controlling access to the data from outside ✓. (2)

8.2 Definition: class – a blueprint / template ✓; object – an instance created from a class ✓. Purpose: class – defines the attributes and methods ✓; object – holds actual values and is used in the program ✓. Example: class Robot ✓; r1 = Robot("Rover") ✓. (6)

8.3 class Sensor: ✓; def __init__(self, name, pin): ✓✓; self.name = name ✓; self.pin = pin ✓; self.value = 0 ✓; correct indentation ✓ (7)

```
class Sensor:
    def __init__(self, name, pin):
        self.name = name
        self.pin = pin
        self.value = 0
```

8.4 temp1 = Sensor("Temperature", 4) ✓✓; print(temp1.name) ✓ (3)

[18]

TOTAL SECTION B: [80]

TOTAL: 120

END OF MARKING GUIDELINE